

Object-Oriented Programming

Inheritance

PBO Team

Inheritance

- Basic Understanding of Inheritance
 - Declaration of Inheritance
 - Single Inheritance
 - When to apply inheritance?
 - Accessing members from the parent class
 - Access control
 - Super keyword
 - Constructors are not inherited
- 

Basic Understanding of Inheritance

- The concept of inheritance is adopting the real world, where objects can have descendants.
- With the concept of inheritance, a class can have a child class.
- A class that has a child class is called a parent class, or base class, or super class.
- The child class is also called sub class.
- A subclass inherits all the class members that exist in its super class (properties, behaviour, etc.) except for private members and constructors.
- In conclusion, a subclass is an extension (extends) of its parent class.

When to Use Inheritance?

- For example, in a game, there is a character that has attributes Name, Level, Health, and method attack()
- In that game, there are several types of characters, namely Angel, Human, and Wizard. Each character has the same characteristics, but there are some characteristics that differentiate them:
 - **Angel has Potion attribute and method cure()**
 - **Human has Strength attribute and method specialAttack()**
 - **Wizard has Spell attribute and method magic()**
- However, all these characters still have Name, Level, and Health attributes.
- Observe the following class diagram:

When to Use Inheritance?

Human
+ Name: String + Level: int + Health: int + Strength: int
+ attack() + specialAttack()

Angel
+ Name: String + Level: int + Health: int + Potion: int
+ attack() + cure()

Wizard
+ Name: String + Level: int + Health: int + Spell: int
+ attack() + magic()

If observed, the classes above have several attributes and methods in common.

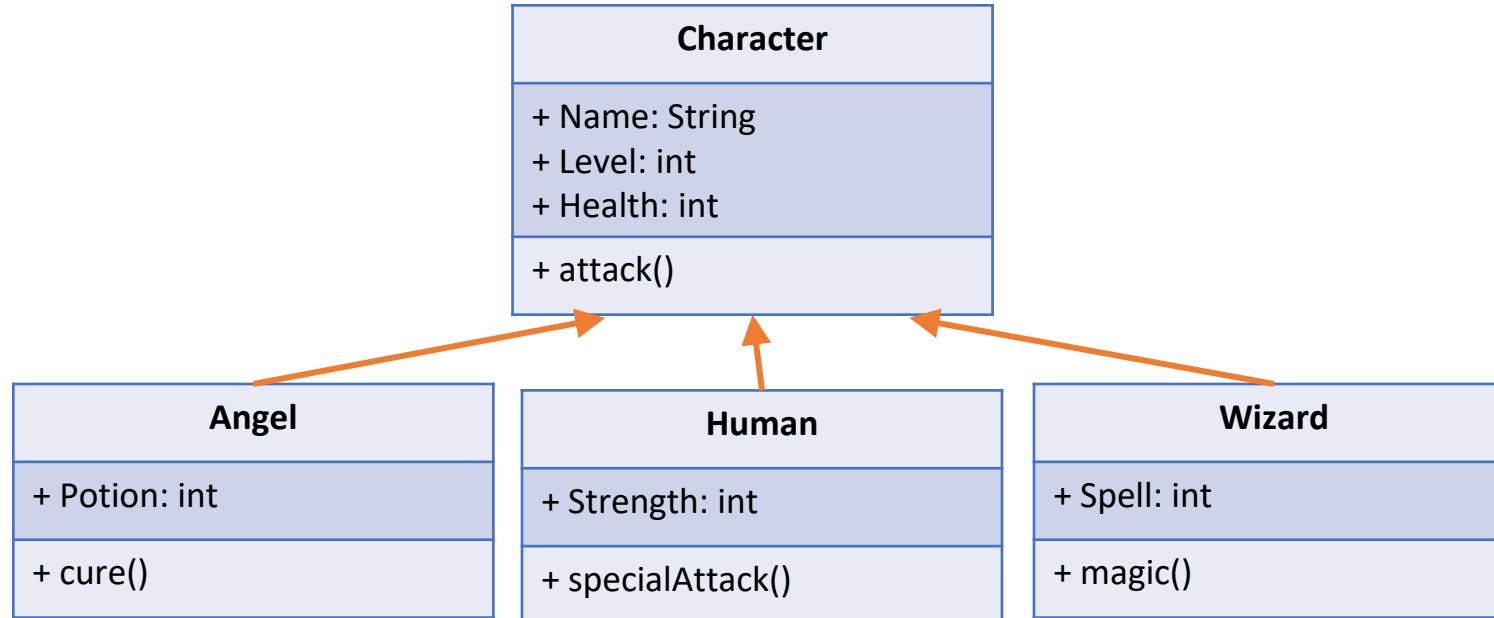
- Thus, there is a repetition of program code.

The solution?

- By utilizing inheritance, we can create a parent class Character that has certain attributes and methods that can be passed down to other classes.
- Thus reducing code repetition.

Observe the following class diagram:

When to Use Inheritance?



- By using inheritance, we create a parent class **Character** that has attributes `Name`, `Level`, `Health`, and method `attack()`
- The attributes and methods of the **Character** class are inherited by child classes, namely **Angel**, **Human**, and **Wizard**.

Declaration of Inheritance

- To declare a class as a subclass, you do so by adding the keyword `extends` after the class name declaration, followed by the name of the parent class.

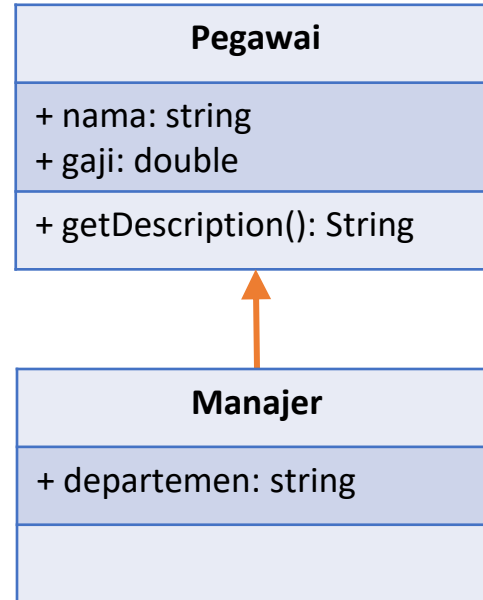
```
public class B extends A
{
    ...
}
```

- The example above informs the Java compiler that class B is a subclass of class A.

Single Inheritance

- Java only allows for single inheritance.
 - This means that a subclass can only have one parent class. However, a parent class can have many subclasses.
- 

Example 1



- When the Manager class inherits from the Employee class, it inherits all member classes owned by the Employee class.
- Thus, the Manager class has all member classes inherited from Employee (name, salary, getDescription), plus the member class it possesses (department).

Example 1 (continued)

```
public class Pegawai
{
    public String nama;
    public double gaji;

    public String getDescription()
    {
        return "Nama: " + nama + ", gaji: " + gaji;
    }
}
```

```
public class Manajer extends Pegawai
{
    public String departemen;
}
```

```
public class TestPegawai
{
    public static void main(String[] args)
    {
        Manajer man = new Manajer();

        man.nama = "Bob";
        man.gaji = 9999999;
        man.departemen = "IT";

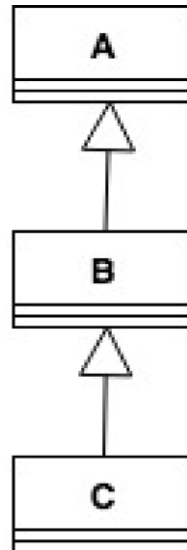
        System.out.println("Nama: " + man.nama);
        System.out.println("Gaji: " + man.gaji);
        System.out.println("Departemen: " + man.departemen);
    }
}
```

```
E:\Java>java TestPegawai
Nama: Bob
Gaji: 9999999.0
Departemen: Bob
```

An object of the Manager class accesses the attributes name and salary, which are inherited from the Employee class.

Multilevel Inheritance

- A subclass can become a superclass for another class.



- It can be seen that B is a subclass of A, and C is a subclass of B.

The keyword this and super (1)

- The keyword this is used to refer to the members of the class itself.
- Meanwhile, the keyword super refers to members of the superclass.
- Example of the keyword this

```
public class Siswa
{
    private int nrp;

    public void setNrp(int nrp)
    {
        this.nrp = nrp;
    }
}
```

The keyword this and super (2)

- Example of the keyword super

```
public class Parent
{
    public int x = 5;
}

public class Child extends Parent
{
    public int x = 10;

    public void Info(int x)
    {
        System.out.println("Nilai x sebagai parameter = " + x);
        System.out.println("Property x di class Child = " + this.x);
        System.out.println("Property x di class Parent = " + super.x);
    }
}
```

The keyword this and super (3)

```
public class TestProgram
{
    public static void main(String args[])
    {
        Child tes = new Child();
        tes.Info(20);
    }
}
```

- Execution result:

```
D:\Java\Inheritance>java TestProgram
Nilai x sebagai parameter = 20
Property x di class Child = 10
Property x di class Parent = 5
```

Access Control (1)

- Members declared as private cannot be inherited.
- Observe the example below:

```
public class Pegawai
{
    private String nama;
    public double gaji;
}

public class Manajer extends Pegawai
{
    public String departemen;

    public void IsiData(String n, String d)
    {
        nama = n;
        departemen = d;
    }
}
```

Access Control (2)

- When the Manager class above is compiled, an error will occur:

```
D:\Java\Inheritance>javac Manajer.java
Manajer.java:8: nama has private access in Pegawai
        nama = n;
        ^
1 error
```

- This proves that the property name in the Employee class is not inherited by the Manager class, because it is declared as private.

Access Control (3)

- The effect of access modifiers on accessing members of a class.

	Same Class Same Package	Other Class Same Package	SubClass Other Package	Other Class Other Package
public	✓	✓	✓	✓
protected	✓	✓	✓	
default	✓	✓		
private	✓			

Access Control (4)

- Try making changes to the Employee class, change the access modifier of the name property to protected and then recompile the Manager class and see what happens.
 - There should be no errors now, because the access of the name property has been changed to protected.
- 

Constructors are not inherited (1)

- Constructors cannot be inherited by subclasses.
 - However, subclasses can call the parent class constructor using the `super()` keyword.
 - Calling the parent class constructor must be done in the subclass constructor at the top line
- 

Constructors are not inherited (2)

- Example:

```
public class Parent
{
    public Parent()
    {
        System.out.println("Ini adalah konstruktor Parent");
    }
}

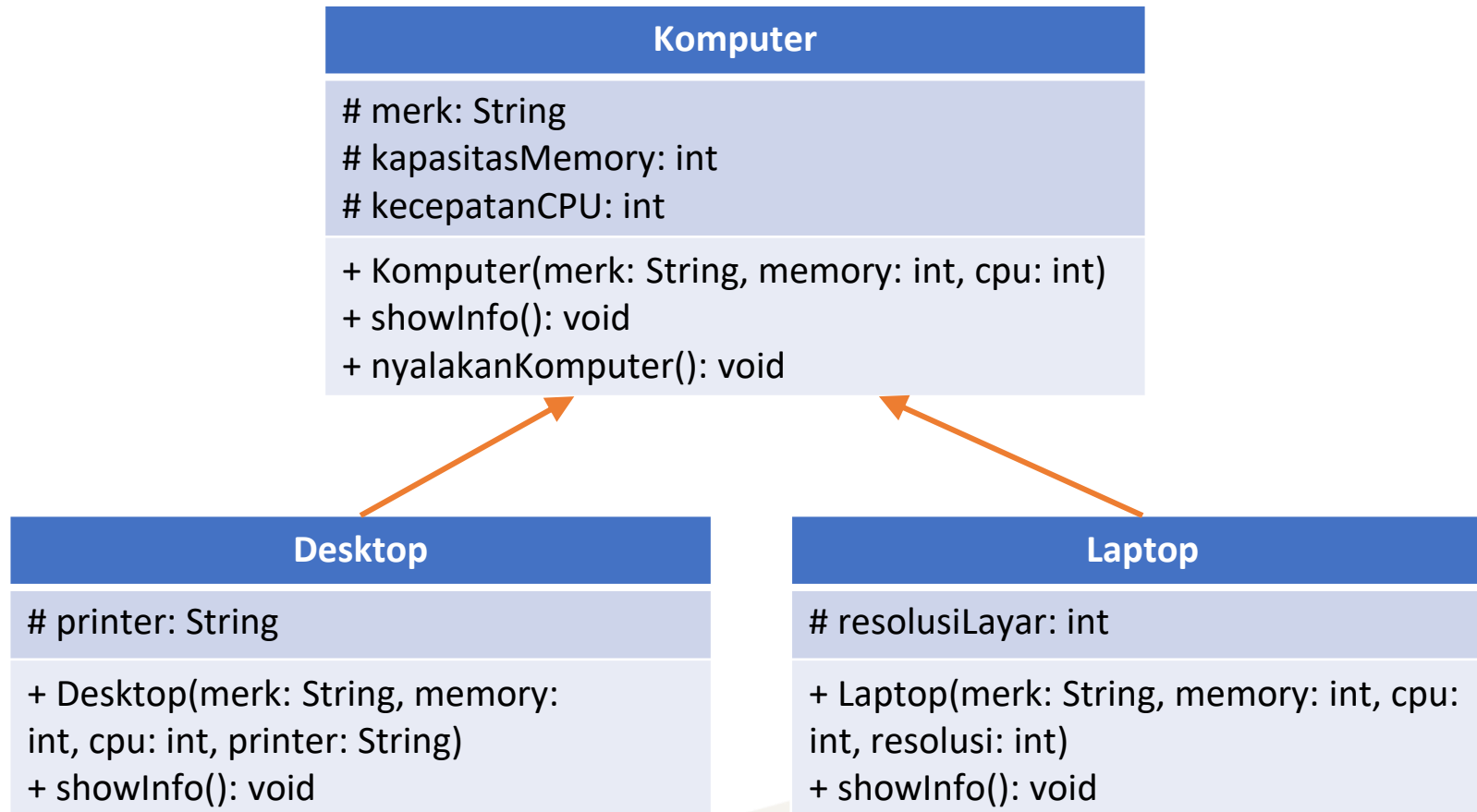
public class Child extends Parent
{
    int x;

    public Child()
    {
        super();
        x = 5;
    }
}
```

Child class Constructor

Calling the Parent constructor
is done in the first line inside
the Child constructor

Example 2



Example 2 (continued)

- Test with the following TestComputer:

```
public class TestComputer
{
    public static void main(String[] args)
    {
        Desktop desk = new Desktop("Dell", 2048, 3500, "Canon");
        Laptop lap = new Laptop("Asus", 4096, 2500, 720);

        desk.showInfo();
        lap.showInfo();

        desk.nyalakanComputer();
    }
}
```

Merk: Dell
Kapasitas Memory: 2048
Kecepatan CPU: 3500
Printer: Canon

Merk: Asus
Kapasitas Memory: 4096
Kecepatan CPU: 2500
Resolusi Layar: 720

Komputer Dell dinyalakan!

Example 2 (continued)

```
public class Komputer
{
    protected String Merk;
    protected int KapasitasMemory;
    protected int KecepatanCPU;

    public Komputer(String merk, int memory, int cpu)
    {
        Merk = merk;
        KapasitasMemory = memory;
        KecepatanCPU = cpu;
    }

    public void showInfo()
    {
        System.out.println("\nMerk: " + Merk);
        System.out.println("Kapasitas Memory: " + KapasitasMemory);
        System.out.println("Kecepatan CPU: " + KecepatanCPU);
    }

    public void nyalakanKomputer()
    {
        System.out.println("\nKomputer " + Merk + " dinyalakan!");
    }
}
```

```
public class Desktop extends Komputer
{
    protected String Printer;

    public Desktop(String merk, int memory, int cpu, String printer)
    {
        super(merk, memory, cpu);
        Printer = printer;
    }

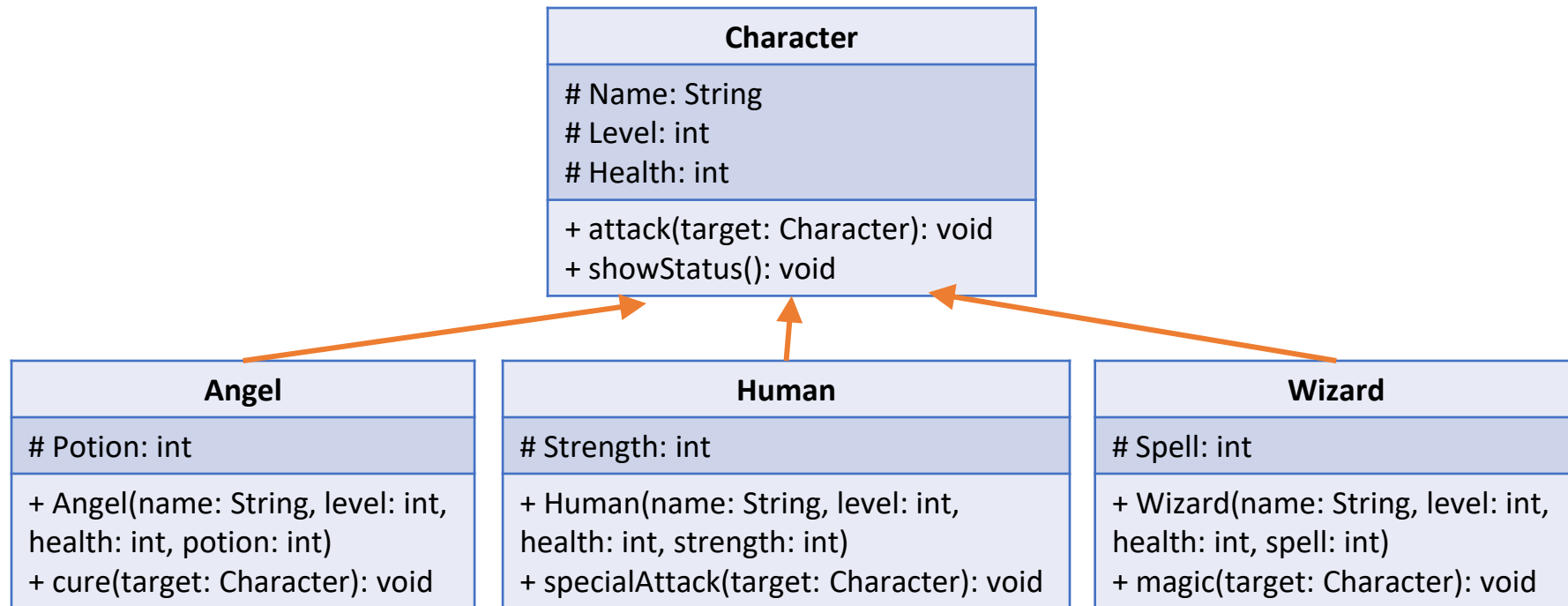
    public void showInfo()
    {
        super.showInfo();
        System.out.println("Printer: " + Printer);
    }
}
```

```
public class Laptop extends Komputer
{
    protected int ResolusiLayar;

    public Laptop(String merk, int memory, int cpu, int layar)
    {
        super(merk, memory, cpu);
        ResolusiLayar = layar;
    }

    public void showInfo()
    {
        super.showInfo();
        System.out.println("Resolusi Layar: " + ResolusiLayar);
    }
}
```

Example 3



- All characters have an attack() method that reduces health by 10.
- Angel has a cure() method that functions to heal, which restores Health to 100. The effect is to reduce the potion.
- Human has a specialAttack() method that reduces the character's Health by 10 + the Strength they possess. It does not affect Strength reduction.
- Wizard has a magic() method that reduces the character's Health by 50. The effect is to reduce Spell.

Example 3 (continued)

- Test with the test program:

```
public class TestAdventureGame
{
    public static void main(String[] args)
    {
        Angel esther = new Angel("Esther", 10, 100, 5);
        Human jackal = new Human("Jackal", 13, 100, 7);
        Wizard quistis = new Wizard("Quistis", 20, 100, 3);

        System.out.println("Begin game...");

        esther.showStatus();
        jackal.showStatus();
        quistis.showStatus();

        System.out.println("Jackal special attack to quistis, "
            + "quistis cast magic to jackal, "
            + "esther cure jackal, "
            + "quistis attack esther...");

        jackal.specialAttack(quistis);
        quistis.magic(jackal);
        esther.cure(jackal);
        quistis.attack(esther);

        esther.showStatus();
        jackal.showStatus();
        quistis.showStatus();

    }
}
```

Output yang diharapkan:

```
Begin game...
Name: Esther, Health: 100
Name: Jackal, Health: 100
Name: Quistis, Health: 100
Jackal special attack to quistis, quistis cast magic to jackal,
esther cure jackal, quistis attack esther...
Name: Esther, Health: 90
Name: Jackal, Health: 100
Name: Quistis, Health: 83
```

Example 3 (continued)

```
public class Character
{
    protected String Name;
    protected int Level;
    protected int Health;

    public void attack(Character target)
    {
        target.Health -= 10;
    }

    public void showStatus()
    {
        System.out.println("Name: " + Name + ", Health: " + Health);
    }
}
```

```
public class Human extends Character
{
    protected int Strength;

    public Human(String name, int level, int health, int strength)
    {
        Name = name;
        Level = level;
        Health = health;
        Strength = strength;
    }

    public void specialAttack(Character target)
    {
        target.Health = target.Health - 10 - Strength;
    }
}
```

```
public class Angel extends Character
{
    protected int Potion;

    public Angel(String name, int level, int health, int potion)
    {
        Name = name;
        Level = level;
        Health = health;
        Potion = potion;
    }

    public void cure(Character target)
    {
        target.Health = 100;
        Potion -= 1;
    }
}
```

```
public class Wizard extends Character
{
    protected int Spell;

    public Wizard(String name, int level, int health, int spell)
    {
        Name = name;
        Level = level;
        Health = health;
        Spell = spell;
    }

    public void magic(Character target)
    {
        target.Health -= 50;
        Spell -= 1;
    }
}
```

Example 4

- There is a `Rekening` (Account) class with a `Saldo` (Balance) property. It includes a `getSaldo` method that returns the balance value, a `simpanUang` method that increases the balance by the deposited amount, and an `ambilUang` method that decreases the balance by the withdrawn amount.
- There is a `Deposito` (Time Deposit) class, which is a subclass of `Rekening`. This class has `BungaPerBulan` (Monthly Interest Rate) and `Tenor` (deposit term in months) properties.
- The constructor accepts initial balance and tenor parameters, assigning the initial balance to the `Saldo` property of the parent class (`Rekening`). The constructor also sets a default `BungaPerBulan` value of 5%. The class includes a `getNilaiBunga` method that calculates the interest value using the following formula:

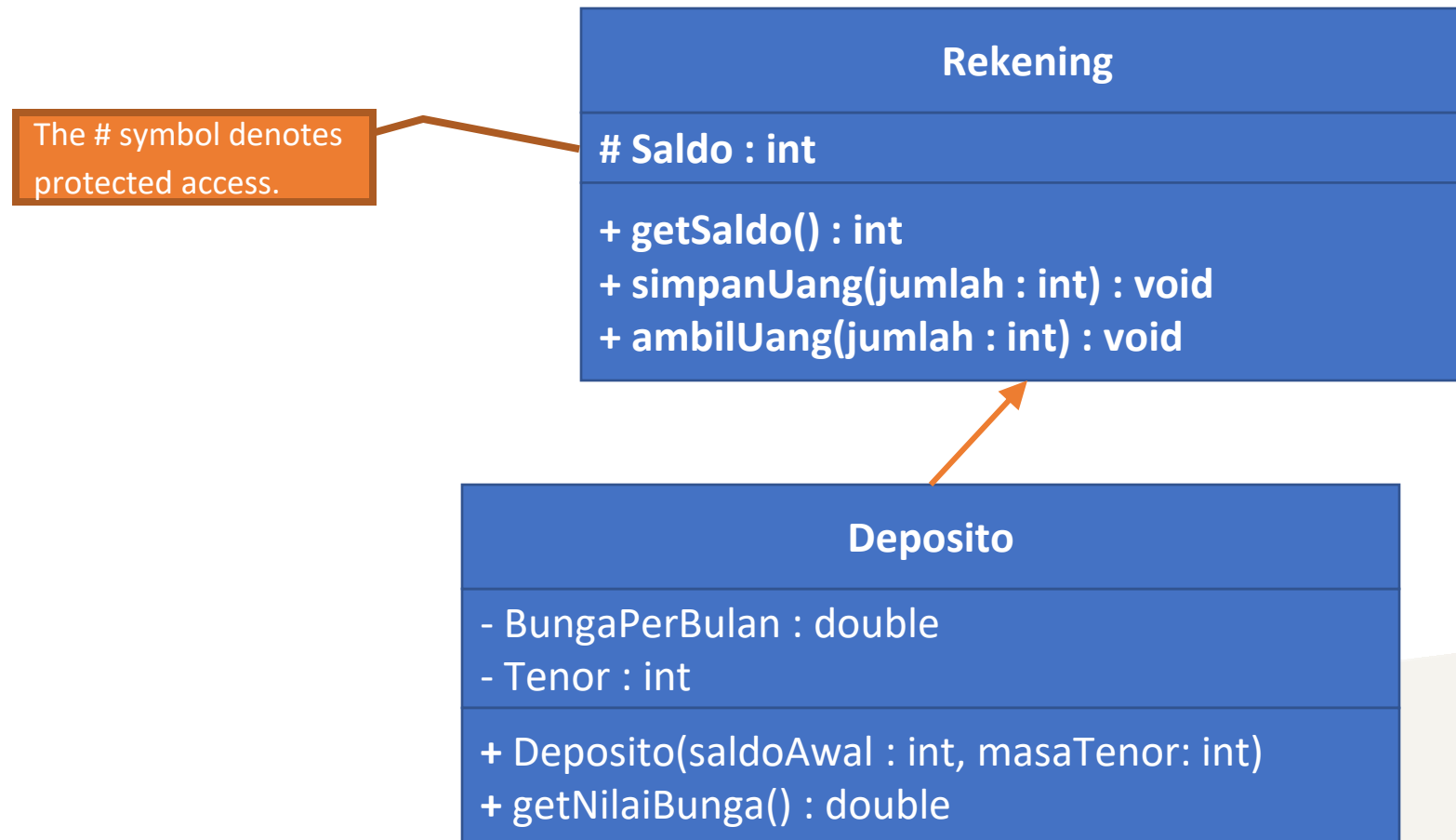
$$\text{Interest Value} = \text{Balance} * \text{Monthly Interest Rate} * \text{Tenor} / 12$$

- For example, if a balance of 1,000,000 is placed in a time deposit with a 36-month tenor and 1% interest, the interest value is:

$$\begin{aligned}\text{Nilai bunga} &= 1.000.000 * 1/100 * 36/12 \\ &= 30.000\end{aligned}$$

Example 4 (continued)

- Class Diagram dari deskripsi diatas



Example 4 (continued)

- Class Rekening

```
public class Rekening
{
    protected int Saldo;

    public int getSaldo()
    {
        return Saldo;
    }

    public void simpanUang(int jumlah)
    {
        Saldo = Saldo + jumlah;
    }

    public void ambilUang(int jumlah)
    {
        Saldo = Saldo - jumlah;
    }
}
```

saldo dibuat
protected agar dapat
diakses oleh class
turunannya

Example 4 (continued)

- Class Deposito

```
public class Deposito extends Rekening
{
    private double BungaPerBulan;
    private int Tenor;

    public Deposito(int saldoAwal, int masaTenor)
    {
        Saldo = saldoAwal;
        Tenor = masaTenor;
        BungaPerBulan = 0.05;
    }

    public double getNilaiBunga()
    {
        double nilaiBunga;

        nilaiBunga = Saldo * BungaPerBulan * Tenor / 12;

        return nilaiBunga;
    }
}
```

Mengakses property
saldo milik class
parent (Tabungan)

Example 4 (continued)

- Class TestDeposito untuk pengetesan:

```
public class TestDeposito
{
    public static void main(String[] args)
    {
        Deposito dpt = new Deposito(1200000, 4);

        System.out.println("Saldo awal: " + dpt.getSaldo());
        System.out.println("Nilai bunga deposito: " + dpt.getNilaiBunga());
        System.out.println("Total deposito: " + (dpt.getSaldo() + dpt.getNilaiBunga()));
    }
}
```

- Output:

```
D:\Java>java TestDeposito
Saldo awal: 1200000
Nilai bunga deposito: 20000.0
Total deposito: 1220000.0
```

Exercise 1

- There are the Televisi and TelevisiModern classes.



Exercise 1 (continued)

- Create the Television and TelevisiModern classes according to the class diagram above.
- Then TestTelevisi for testing:

```
public class TestTelevisi
{
    public static void main(String[] args)
    {
        TelevisiModern tv = new TelevisiModern("Samsung", 100);
        System.out.println("Channel aktif: " + tv.getChannelAktif());
        tv.pindahChannel(20);
        System.out.println("Channel aktif sekarang: " + tv.getChannelAktif());
        tv.gantiModusTampilan("HDMI");
        tv.mainkanDVD();
        tv.masukkanDVD("The Matrix");
        tv.mainkanDVD();
    }
}
```

```
D:\Java\Inheritance>java TestTelevisi
Channel aktif: 1
Channel aktif sekarang: 20
Sedang memainkan DVD: kosong
Sedang memainkan DVD: The Matrix
```

To be continued...